

STUDENT 2

LLM API, Agent Brain and Organizations

Connect the agents to an LLM and make swarm and centralized organizations behave differently.

ABMS-LLM Cyber Organizational Simulation

Beginner work plan for implementing LLM agents in the current repository. This document is written for both the assigned student and an AI coding assistant.

Main research question

How does agent behavior and performance differ when LLM-driven cyber agents are organized as an independent swarm versus a centralized hierarchy?

Important decision

- LLM cognition is now the main implementation target.
- The existing rule-based cognition stays small and working only as a baseline and API-failure fallback.
- Do not delete the rule baseline, because the research needs a control and developers need a cheap debugging mode.
- Work in the Conda environment named mesa and use Mesa 3.x APIs.

1. Your Job in Simple Words

You are the main LLM student. You own the API client, prompts, response validation, agent cognition, memory and organization logic. Your code asks an LLM for a decision, checks the answer, and sends a safe action to Student 1's resolver.

How one LLM decision works

- 1 Student 1 turns the current simulation state into a small observation dictionary.
- 2 Student 2 sends that observation to the LLM with instructions and allowed actions.
- 3 The LLM returns one JSON decision. Student 2 checks it before trusting it.
- 4 Student 1's resolver applies the valid action and returns the result.
- 5 Student 3 records behavior, outcome, tokens, time, errors and estimated cost.
- 6 Student 4 shows safe decision summaries and metrics in the UI and documentation.

Words you need to know

Word	Simple meaning
LLM	A language model that reads an observation and chooses an action.
API	The online service used by Python to ask the LLM for a response.
Prompt	The instructions and simulation information sent to the LLM.
JSON	A strict text format used so Python can safely read the LLM answer.
Token	A small unit of text used to measure API usage and cost.
Fallback	A safe rule-based action used when the API or LLM answer fails.
Mock client	A fake API used in tests so tests are fast, free and repeatable.

2. Files You Own

File	What it should do
abms_cyber/agents/cognition/llm_cognition.py	Create the LLM brain that implements BaseCognition.
abms_cyber/agents/cognition/llm_client.py	Create one small API wrapper so provider code is not spread everywhere.
abms_cyber/agents/cognition/prompts.py	Store system prompts and prompt-building functions.
abms_cyber/agents/cognition/response_parser.py	Validate JSON and turn it into one safe action.
abms_cyber/agents/cyber_agent.py	Use cognition, execute the chosen action and update memory.
abms_cyber/agents/memory.py	Store local observations, decisions, outcomes and short history.
abms_cyber/organization/centralized.py	LLM leader assigns actions to workers.
abms_cyber/organization/swarm.py	Each agent independently asks the LLM.

What you must build

- A provider wrapper configured by environment variables. Never save an API key in Python, Git, a PDF or a log.
- LLMCognition implementing the existing BaseCognition idea, using observations from Student 1.
- A short prompt that demands one JSON action and forbids invented actions/targets.
- Strict parsing and validation. Bad JSON, timeout, rate limit or invalid action must not crash the simulation.
- Swarm mode: each agent decides independently. Centralized mode: one LLM leader receives the organization view and assigns worker actions.
- A small rule-based fallback for API failure and a rule-based baseline for experiments; do not spend most of the project improving it.

Stay inside your role

Do not redesign another student's layer without agreement. If you need a new field or function, first write a small example of the input and output, then ask its owner to approve the interface.

3. Shared LLM Data Format

All students should use one simple data format. Student 1 builds the observation. Student 2 sends it to the LLM and validates the response. Students 3 and 4 read safe metadata from the result.

Example observation

```
{
  "agent_id": 1,
  "organization": "swarm",
  "current_node": "3",
  "privilege_level": 0,
  "known_nodes": ["3", "4"],
  "recent_actions": ["SCAN:3"],
  "detection_level": 0.20,
  "goal": "Compromise the target with admin privilege",
  "allowed_actions": ["SCAN", "MOVE", "EXPLOIT", "ESCALATE"]
}
```

Example LLM response

```
{
  "action": "EXPLOIT",
  "target_id": "3",
  "reason": "The current node is scanned and not compromised."
}
```

Safety rules

- Never execute Python code returned by the LLM.
- Never trust action or target text until it is checked against allowed values.
- Never hardcode, commit, print or display the API key.
- Use environment variables for secrets and fake clients for normal tests.
- Keep prompts small enough to control cost and store only safe logs.

4. Build It Step by Step

- 1 Choose the LLM provider with the supervisor. Put model name, timeout and retry count in configuration; read the secret key from an environment variable.
- 2 Create `llm_client.py` with one method that accepts messages and returns text plus usage information. Mock this method in tests.
- 3 Create `prompts.py`. Tell the model its goal, show allowed actions and require the exact JSON response format.
- 4 Create `response_parser.py`. Reject extra actions, missing fields, unknown targets and text that is not valid JSON.
- 5 Create `LLMCognition.decide()`. It builds the prompt, calls the client, validates the response and returns one action/result object.
- 6 Implement swarm flow: every agent receives its own observation and calls `LLMCognition` independently.
- 7 Implement centralized flow: one selected leader receives the global organization view and returns one assignment per worker; workers do not call independent cognition in that step.
- 8 Add timeout/retry handling, then fallback to the simple rule policy when the API still fails. Mark fallback decisions in logs.
- 9 Send usage, latency, error and decision data to Student 3's metrics interface.

When using an AI coding assistant

- Give the assistant this PDF and the repository files named in your assignment.
- Ask it to inspect before editing and to explain what already exists.
- Request one small task at a time; review the diff before moving to the next task.
- Require tests with a mock API before making a paid live request.
- Do not accept code that hides errors, exposes secrets or changes unrelated files.

5. Tests and Definition of Done

Your work is done only when these checks pass:

- Tests use a fake API client and never spend real tokens.
- Valid JSON creates exactly one allowed action.
- Bad JSON, missing fields and invented actions are rejected safely.
- A timeout or API error retries only the configured number of times and then uses fallback.
- Swarm agents make separate calls and keep separate memory.
- Centralized mode selects one leader and workers execute its assignments without making extra independent calls.
- API keys never appear in logs, exceptions, prompts saved to disk or Git files.

Minimum evidence to submit

- List of files changed and one sentence explaining each file.
- Automated test output for normal, invalid and failure behavior.
- One end-to-end mocked-LLM run that costs nothing.
- One carefully limited live-LLM smoke run if the team has an API key and budget.
- A note listing remaining limitations and any assumptions agreed by the team.

Cost warning

Do not begin with many trials. Start with mocked responses, then one live step, then one short episode. Student 3 must add call/token/cost limits before the team runs large experiments.

6. Six-Week Plan and Team Handoffs

Week	Main result
1	Agree provider, action JSON, observation JSON and organization behavior.
2	Build mockable API client, prompt builder and response parser.
3	Build LLM Cognition with retries, validation and fallback.
4	Complete swarm and centralized LLM flows.
5	Integrate usage metrics, run small paid tests and control failures.
6	Prompt tuning, documentation and final behavior demonstration.

Required team handoffs

- Student 1 gives Student 2 the observation fields and action-resolver interface.
- Student 2 gives Student 3 sanitized LLM usage events and gives Student 4 safe decision-log fields.
- Student 3 gives Student 4 stable DataCollector column names and termination status.
- Student 4 gives everyone the final runbook and reports integration defects to the correct owner.

Four contracts to agree in Week 1

- Observation JSON fields.
- LLM response/action JSON fields.
- One model-step sequence for swarm and centralized modes.
- LLM usage and decision-log fields for metrics and UI.

7. Prompt to Give an AI Coding Assistant

Attach or provide this PDF and then send the following prompt. The assistant must still inspect the real repository because this plan describes responsibilities, not the final code state.

You are helping Student 2 on AbstractCyberSim. You own LLM API integration, cognition, memory and swarm/centralized organization behavior. First read Knowledge/Requierment, Knowledge/Important rules, the agent/cognition and organization folders, and Student 1's observation/action interfaces. Use Mesa 3.x. Build a provider-isolated, mockable LLM client; LLMCognition; concise prompts; strict JSON response validation; retries; safe fallback; and tests that use a fake client. Swarm means independent LLM decisions. Centralized means one LLM leader assigns worker actions and workers do not independently decide in the same step. Never hardcode or print API keys. Explain changes for a beginner and run tests before finishing.

Ask the AI to finish each reply with

- What it changed in beginner-friendly language.
- Which files it changed.
- Which tests and commands it ran and their results.
- Any unresolved problem or decision needed from another student.
- Whether any live API call was made and its approximate usage/cost.

Final reminder

The goal is not merely to make an API call. The goal is to create a controlled research system where valid LLM decisions can be compared fairly between swarm and centralized organizations.